

好的，我會一句一句、區塊區塊詳細教你看懂這支 Stored Procedure。

這支 SP 的名稱是 GLSPEC_SyncDJToInternalList，主要功能是：

把 Dow Jones (DJ) 的制裁名單同步到內部兩個不同的制裁名單表格：

1. 主要黑名單 → BSADB.dbo.OFAC_GLSpec (根據傳入的 @GroupId)
2. 制裁控制與所有權名單 (SCO) → BSADBTW.dbo.OFAC_SPEC (固定寫到 Group 27)

第一區塊：開頭 + 參數 + 設定變數

```
SQL

USE [BSADB]
GO
ALTER PROCEDURE [dbo].[GLSPEC_SyncDJToInternalList]
@GroupId BIGINT
AS
BEGIN

DECLARE @OtherIdentMthCode VARCHAR(2),
        @Filter_Alt_Str INT

SELECT @OtherIdentMthCode = IDENTCODE
FROM dbo.IDENTMTHD_DIM
WHERE OTHIND = 'Y'

SELECT @Filter_Alt_Str = CtrlVal
FROM dbo.PO_DIMData
WHERE Module = 'DJ' AND CtrlKey = 'DJ_ToGLSPEC_ALT_STR'
```

解釋：

- USE [BSADB]：這支 SP 預設在 BSADB 這個資料庫執行。
- @GroupId：傳入參數，決定要把資料 sync 到 OFAC_GLSpec 的哪一個 Group (因為同一個表格可能有多個不同的制裁名單群組)。
- @OtherIdentMthCode：從 IDENTMTHD_DIM 找出「其他識別方式」的代碼 (通常是護照、身分證以外的證件類型)。

- @Filter_Alt_Str：從參數設定檔 PO_DIMData 讀取「AltStrength (替代強度)」的過濾條件。這是用來控制「要不要把某些弱關聯的別名也納入」的開關 (很重要！)。

第二區塊：建立暫存表 (Main Blacklist)

```
SQL

IF OBJECT_ID('tempdb..#tempDJBlacklist') IS NOT NULL
    DROP TABLE #tempDJBlacklist
IF OBJECT_ID('tempdb..#tempDJBlacklistDetails') IS NOT NULL
    DROP TABLE #tempDJBlacklistDetails

CREATE TABLE #tempDJBlacklist (
    EntNum INT NOT NULL,
    SdnName NVARCHAR(350) NOT NULL,
    UpdatedDate VARCHAR(8) NOT NULL
)

CREATE TABLE #tempDJBlacklistDetails(
    EntNum INT NOT NULL,
    SdnName NVARCHAR(350) NOT NULL,
    UpdatedDate VARCHAR(8) NOT NULL,
    Address NVARCHAR(400),
    City NVARCHAR(200),
    Country NVARCHAR(100),
    IdentType VARCHAR(2),
    IdentNumber NVARCHAR(44),
    DOB VARCHAR(8),
    Gender NVARCHAR(7)
)
```

為什麼要用暫存表？

因為 DJ 的資料分散在很多張表 (DJ_List_Data_A 、 DJ_Address 、 DJ_IdentificationTypeInfo ...)，所以先把「核心名單」跟「詳細資料」整理到暫存表，之後再一次 MERGE 進目標表，比較好控制與除錯。

第三區塊：取得主要黑名單 (#tempDJBlacklist)

SQL

```
INSERT INTO #tempDJBlacklist (EntNum, SdnName, UpdatedDate)
SELECT A.EntNum, A.SdnName, MAX(A.UpdatedDate) AS UpdatedDate
FROM BSADB.dbo.DJ_List_Data_A A
WHERE A.NamePropertyBitCode & 1 > 0
      AND A.AltStrengthBitCode & @Filter_Alt_Str <> 0
GROUP BY A.EntNum, A.SdnName
```

重點解釋：

- NamePropertyBitCode & 1 > 0：只取「主要名稱」 (Primary Name)。Bit 0 = 1 代表這是主要名字。
- AltStrengthBitCode & @Filter_Alt_Str <> 0：根據系統參數 @Filter_Alt_Str 過濾「替代名稱強度」。這是為了避免把太弱的別名也塞進來，影響命中精準度。
- MAX(A.UpdatedDate) + GROUP BY：同一個 EntNum 可能有多筆更新，取最新的一筆。

第四區塊：用 CTE 取得地址、證件、出生日期、性別（只取第一筆）

SQL

```
;WITH TempAddress AS (
    SELECT A.EntityId, A.Address, A.City, A.AddressCountry,
           ROW_NUMBER() OVER(PARTITION BY A.EntityId ORDER BY A.Id) AS
    'RowId'
    FROM BSADB.dbo.DJ_Address A WITH(NOLOCK)
    WHERE A.Deleted = 0
          AND EXISTS(SELECT 1 FROM #tempDJBlacklist X WHERE A.EntityId =
X.EntNum)
),
TempIdent AS (...),
TempDOB AS (...),
TempGender AS (...)
```

為什麼用 ROW_NUMBER() + PARTITION BY？

因為一個人可能有多個地址、多個證件。我們只想要「第一筆」（最主要的那一筆），所以用 ROW_NUMBER() OVER(PARTITION BY EntityId ORDER BY Id)，之後只取 RowId = 1。

這是處理「一對多」關聯時很常見的寫法。

第五區塊：把詳細資料組起來 (#tempDJBlacklistDetails)

SQL

```
INSERT INTO #tempDJBlacklistDetails (...)  
SELECT A.EntNum, A.SdnName, A.UpdatedDate,  
       B.Address, LEFT(B.City,100), LEFT(B.AddressCountry,100),  
       C.IdentificationType, LEFT(C.IdentificationNumber,44),  
       CASE WHEN D.InfoDay IS NULL OR D.InfoMonth IS NULL  
            THEN D.InfoYear  
            ELSE CONVERT(VARCHAR(8), dbo.TryParseDate(...), 112) END,  
       E.Gender  
FROM #tempDJBlacklist A  
LEFT JOIN TempAddress B ON A.EntNum = B.EntityId AND B.RowId = 1  
LEFT JOIN TempIdent C   ON A.EntNum = C.EntityId AND C.RowId = 1  
...
```

這裡就是把剛剛 CTE 裡整理好的地址、證件、生日、性別 **LEFT JOIN** 回來，組成一筆完整的名單資料。

第六區塊：核心 MERGE (寫入 OFAC_GLSpec + 記 Log)

SQL

```
;WITH TargetTable AS (  
    SELECT ... FROM BSADB.dbo.OFAC_GLSpec A  
    WHERE A.GROUPID = @GroupId  
)  
MERGE TargetTable T  
USING (SELECT ... FROM #tempDJBlacklistDetails) S  
ON T.spec_name = S.SdnName AND T.remark = CAST(S.EntNum AS VARCHAR(100))
```

MERGE 的三種情況：

情況	說明	動作
WHEN NOT MATCHED BY TARGET	來源有、目標沒有	INSERT 新資料， DInd='N'

WHEN NOT MATCHED BY SOURCE AND T.DInd='N'	目標有、來源沒有，且目前是有有效的	Soft Delete → DInd='Y'
WHEN MATCHED AND (T.DInd='Y' OR S.UpdatedDate <> T.updated_date)	兩邊都有，但目標被刪除過，或資料有更新	恢復 + 更新 資料

為什麼用 Soft Delete (DInd='Y') ?

因為制裁名單不能真的刪除，必須保留歷史軌跡，所以用 DInd 欄位來標記「已刪除」。

最後的 OUTPUT ... INTO BSADB.dbo.OFAC_GLSpec_Log 是把每一次的 Insert/Update/Delete 都記錄到 Log 表，包含 ActionCD (01=新增、02=更新、03=刪除)。

第七區塊：SCO (Sanctions Control & Ownership) 部分

從這裡開始是第二段邏輯 (2025/12/19 新增的)：

SQL

```
--20251219 IT20202509021049 split Sanctions Control & Ownership list from  
dowjones
```

這段是把「被制裁公司的大股東 / 控制人」這類名單，另外同步到 BSADBTW 資料庫的 OFAC_SPEC 表格 (GroupId = 27)。

重點不同之處：

1. 資料來源條件更嚴格：

- 只取 DJ_ListCategory.Description3 是以下類別的：
 - 'OFAC Related - Majority Owned'
 - 'OFAC Related - Control'
 - 'EU Related - Majority Owned' 等

2. 需要產生新的 ent_num :

SQL

```
SELECT @MAXNo = MAX(ent_num) FROM BSADBTW..OFAC_SPEC
UPDATE #tempDJBlacklistDetailsSCO
SET @MAXNo = @MAXNo + 1, Num = @MAXNo
WHERE Num IS NULL
```

因為這是寫到另一個資料庫的表格，所以要自己算新的序號。

3. SanctionListReference 的處理：

用 STUFF + FOR XML PATH 把同一個 Entity 的多個 Reference 串成用 ; 分隔的字串。如果超過 400 字就直接寫「Visit the Dow Jones for further information」。

總結這支 SP 的設計重點

重點	說明
暫存表 + CTE	把複雜的多表 JOIN 先整理乾淨
ROW_NUMBER() 取第一筆	處理一對多關係（地址、證件、生日）
Bitwise AND 過濾	NamePropertyBitCode & 1、AltStrengthBitCode
MERGE + Soft Delete	一次搞定新增、更新、logical delete
雙資料庫設計	主要名單在 BSADB，SCO 名單另外寫到 BSADBTW 的 Group 27
完整 Log	所有異動都寫到 Log 表，方便追蹤

想讓我繼續更細部解釋哪一段？

例如：

- 你想我再詳細講 MERGE 的三種情境 + Log 的對應關係？
- 還是想了解 AltStrengthBitCode 這個過濾條件的業務意義？
- 或者 SCO 這段為什麼要獨立寫到 BSADBTW？

告訴我你哪邊比較卡，我再針對性詳細解釋。